AWS Lambda Project Report

(Cron-Based + Event-Based Processing)

1. Project Overview

This project demonstrates two AWS Lambda-based architectures:

1. **Task 1: Cron-Based Processing**
    - Scheduled Lambda function runs daily
    - Aggregates order data per user
2. **Task 2: Event-Based Processing**
    - S3 upload triggers Lambda automatically
    - Filters completed orders from uploaded file

---

2. AWS Services Used

- AWS Lambda
- Amazon S3
- Amazon EventBridge (CloudWatch Events)
- IAM Roles & Policies

---

3. Dataset Description

orders.csv

Column Name	Description
OrderID	Unique order ID
UserID	User placing order
OrderDate	Date of order
Amount	Order amount
Status	Order status (Completed, Pending, Failed)

4. Task 1: Cron-Based Lambda Function

Objective

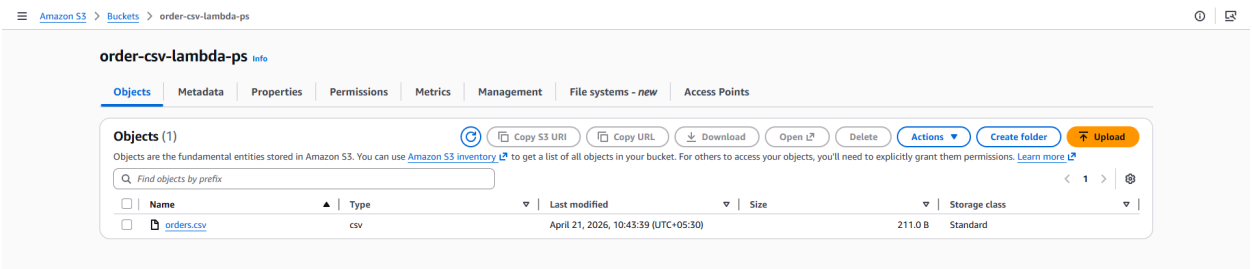
Aggregate number of:

- Completed orders
- Pending orders
- Failed orders for each user daily.

Architecture

- EventBridge Cron Trigger → Lambda → S3 Output

- S3 bucket creation with file upload



Schedule Used

- Midnight UTC:

  cron(0 0 * * ? *)

Schedule pattern

Occurrence | Info

You can define an one-time or recurrent schedule.

- ○ One-time schedule
- ◉ Recurring schedule

Time zone

The time zone for the schedule.

(UTC+05:30) Asia/Calcutta

Schedule type

Choose the schedule type that best meets your needs.

- ◉ **Cron-based schedule**
  A schedule set using a cron expression that runs at a specific time, such as 8:00 a.m. PST on the first Monday of every month.
- ○ **Rate-based schedule**
  A schedule that runs at a regular rate, such as every 10 minutes.

Cron expression | Info

Define the cron expression for the schedule

Copy Clear

cron (	0	0	*	*	?	*	)
	Minutes	Hours	Day of month	Month	Day of the week	Year	

Next 10 trigger dates

Date and time are displayed in your current time zone in UTC format, e.g. "Wed, Nov 9, 2022 09:00 (UTC - 08:00)" for Pacific time

Wed, 22 Apr 2026 00:00:00 (UTC+05:30)
Thu, 23 Apr 2026 00:00:00 (UTC+05:30)
Fri, 24 Apr 2026 00:00:00 (UTC+05:30)
Sat, 25 Apr 2026 00:00:00 (UTC+05:30)
Sun, 26 Apr 2026 00:00:00 (UTC+05:30)
Mon, 27 Apr 2026 00:00:00 (UTC+05:30)
Tue, 28 Apr 2026 00:00:00 (UTC+05:30)
Wed, 29 Apr 2026 00:00:00 (UTC+05:30)
Thu, 30 Apr 2026 00:00:00 (UTC+05:30)
Fri, 01 May 2026 00:00:00 (UTC+05:30)

Select target

Target detail

Target API | Info

Select an API that will be invoked as a target for your schedule.

- ◉ Templated targets
- ○ All APIs

CodeBuild StartBuild	CodePipeline StartPipelineExecution	Amazon ECS RunTask	Amazon EventBridge PutEvents
Amazon Inspector V1 StartAssessmentRun	Kinesis Data Firehose PutRecord	Kinesis Data Streams PutRecord	AWS Lambda Invoke
Amazon SNS Publish	Amazon SQS SendMessage	SageMaker StartPipelineExecution	AWS Step Functions StartExecution

Invoke

Universal target definition

AWS Lambda

Lambda function

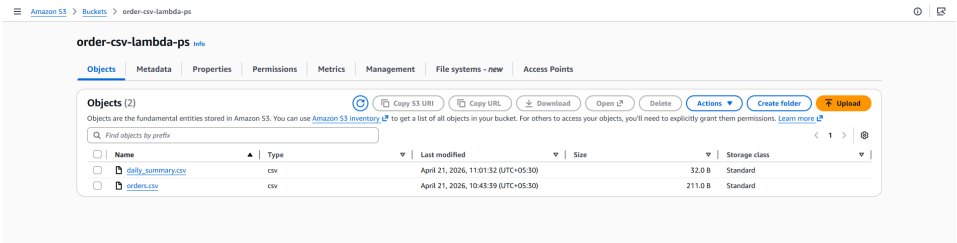
lambda-func-task1-ps

Create new Lambda function

Lambda Code (Task 1)

```python
import csv
import boto3
from collections import defaultdict

s3 = boto3.client('s3')

BUCKET_NAME = "order-csv-lambda-ps"
INPUT_FILE = "orders.csv"
OUTPUT_FILE = "daily_summary.csv"

def lambda_handler(event, context):
    response = s3.get_object(Bucket=BUCKET_NAME, Key=INPUT_FILE)
    content = response['Body'].read().decode('utf-8').splitlines()

    reader = csv.DictReader(content)
    reader.fieldnames = [f.strip().lower() for f in reader.fieldnames]

    print("Normalized Columns:", reader.fieldnames)

    summary = defaultdict(lambda: {
        "completed": 0, "pending": 0,"failed": 0
    })

    for row in reader:

        row = {k.strip().lower(): v.strip() for k, v in row.items()}

        user = row.get('user_id')
        status = row.get('status')

        if not user or not status:
            continue

        status = status.lower()

        if status in summary[user]:
            summary[user][status] += 1

    output = ["user_id,completed,pending,failed"]

    for user, counts in summary.items():
        output.append(
            f"{user},{counts['completed']},{counts['pending']},{counts['failed']}"
        )

    s3.put_object(
        Bucket=BUCKET_NAME,
        Key=OUTPUT_FILE,
        Body="\n".join(output)
    )

    return {
        "statusCode": 200,
        "body": "Aggregation complete"
    }
```

Output Example -



SQL query

Amazon S3 Select supports only the SELECT SQL command. Using the S3 console, you can extract up to 40 MB of records from REST API. For more complex SQL queries, use Amazon Athena

```
/* To create reference point for writing SQL queries, you can display the first 5 records of inpu
SELECT * FROM s3object s LIMIT 5
```

SQL Ln 1, Col 1 Errors: 0 Warnings: 0

Query results

Query results are not available after you choose **Close** or navigate away. Choose **Download results** to download a copy of the f

Status

Successfully returned 5 records in 1034 ms

Bytes returned: 65 B

Raw | **Formatted**

```
user_id,completed,pending,failed
1,1,0,1
2,0,1,0
3,1,0,0
4,1,0,0
```

PROBLEMS OUTPUT CODE REFERENCE LOG TERMINAL Execution Results

```
Status: Succeeded
Test Event Name: test

Response:
{
  "statusCode": 200,
  "body": "Aggregation complete"
}

The area below shows the last 4 KB of the execution log.

Function Logs:
START RequestId: a43f3017-5850-4233-9fc7-6fe55238f607 Version: $LATEST
Normalized Columns: ['orderid', 'userid', 'orderdate', 'amount', 'status']
END RequestId: a43f3017-5850-4233-9fc7-6fe55238f607
REPORT RequestId: a43f3017-5850-4233-9fc7-6fe55238f607  Duration: 403.13 ms Billed Duration: 928 ms Memory Size: 128 MB Max Memory Used: 96 MB  Init
Duration: 524.14 ms

Request ID: a43f3017-5850-4233-9fc7-6fe55238f607
```

5. Task 2: Event-Based Lambda Function

Objective

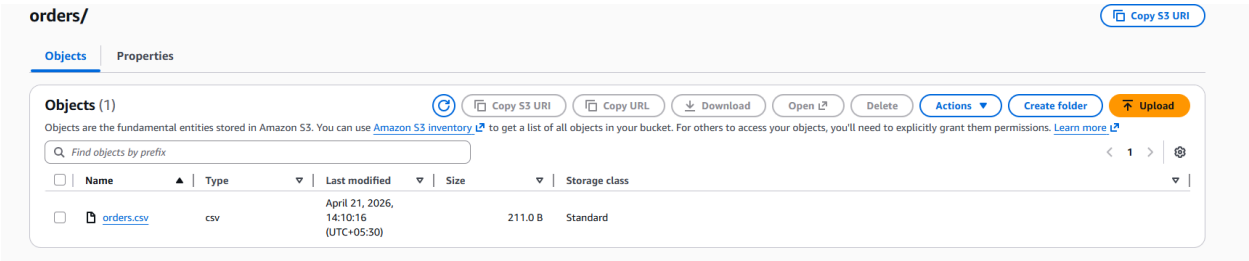
Trigger Lambda when a file is uploaded to S3 and:

- Read file
- Extract only "Completed" orders
- Store filtered results

Architecture

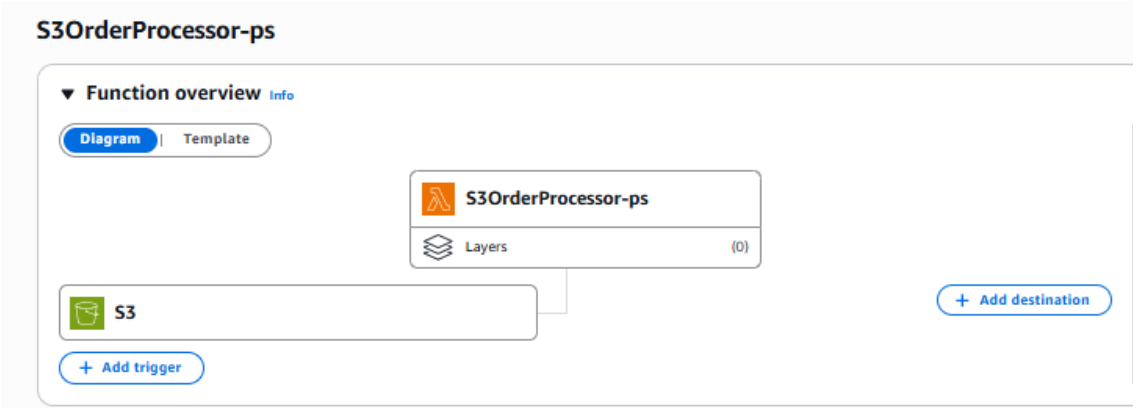
S3 Upload → Lambda Trigger → Filter Data → Store Output

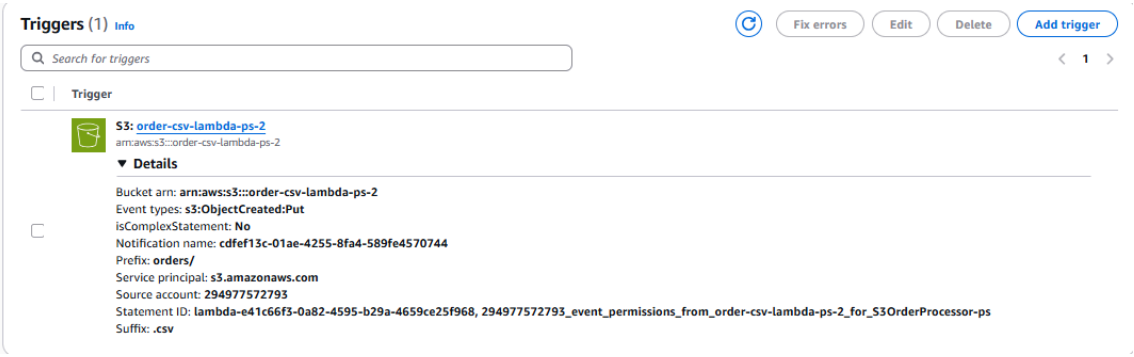
- S3 upload



Trigger Configuration

- Event type: PUT
- Prefix: uploads/
- File type: .csv





Lambda Code (Task 2)

```python
import csv
import boto3
import urllib.parse

s3 = boto3.client('s3')

def lambda_handler(event, context):

    for record in event['Records']:
        bucket = record['s3']['bucket']['name']
        key = urllib.parse.unquote_plus(record['s3']['object']['key'])

        print("File uploaded:", key)

        response = s3.get_object(Bucket=bucket, Key=key)
        content = response['Body'].read().decode('utf-8').splitlines()

        reader = csv.DictReader(content)

        completed_orders = []

        for row in reader:
            row = {k.strip().lower(): v.strip() for k, v in row.items()}

            if row.get('status', '').lower() == 'completed':
                completed_orders.append(row)

        print("Completed Orders:", len(completed_orders))

        output_key = key.replace("orders/", "processed/completed_")

        if completed_orders:
            output = []

            headers = list(completed_orders[0].keys())
            output.append(",".join(headers))

            for order in completed_orders:
                output.append(",".join([order[h] for h in headers]))

            s3.put_object(
                Bucket=bucket,
                Key=output_key,
                Body="\n".join(output)
            )

        return {
            "statusCode": 200,
            "body": f"Processed {key}, completed orders: {len(completed_orders)}"
        }
```

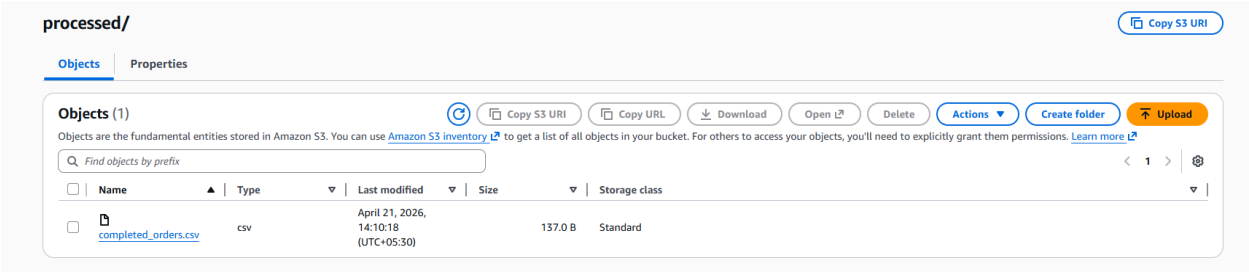
```
PROBLEMS   OUTPUT   CODE REFERENCE LOG   TERMINAL                                   Execution Results

Status: Succeeded
Test Event Name: testlambda

Response:
{
  "statusCode": 400,
  "body": "This Lambda only works with S3 trigger events"
}

The area below shows the last 4 KB of the execution log.

Function Logs:
START RequestId: 96988cd6-f40c-40d8-b51a-be7694997617 Version: $LATEST
END RequestId: 96988cd6-f40c-40d8-b51a-be7694997617
REPORT RequestId: 96988cd6-f40c-40d8-b51a-be7694997617  Duration: 2.14 ms   Billed Duration: 603 ms Memory Size: 128 MB Max Memory Used: 95 MB  Init
Duration: 600.43 ms

Request ID: 96988cd6-f40c-40d8-b51a-be7694997617
```

Output Example

```
completed_orders.csv ×
C: > Users > parthk.shah > Downloads > completed_orders.csv
1   orderid,userid,orderdate,amount,status
2   101,1,2024-05-01,50.00,Completed
3   103,3,2024-05-03,50.00,Completed
4   105,4,2024-05-05,50.00,Completed
```

6. IAM Permissions Required

- AmazonS3FullAccess (or restricted policy)



7. Conclusion

This project demonstrates:

- Event-driven serverless architecture
- Automated scheduling using cron jobs
- Real-time file processing using S3 triggers
- Data aggregation and filtering using AWS Lambda